FAST NATIONAL UNIVERSITY, KARACHI CAMPUS

Department of Software Engineering

BSc. program in Software Engineering

Project Management Plan

IntelliMeet Pro: AI Meeting Assistant & Workflow Management

Lecturer:
Prof. Minhal Raza

Student:
Mustafa Masood 22K-4818
Laiba Fatima 22K-5195
Sabina Rasheed 22K-5198

Date of issue:
20th September, 2025

Baseline version number:
0001

ACADEMIC YEAR 2025/2026

CHANGE HISTORY

PREFACE

This document serves as template of the Project Management Plan (PMP) for the course Fundamentals of Software Project Management (FSPM); reference books: [1, 2]. Note that it is intended for the Academic Year 2025/2026; the author is not responsible for any misuse should the course's regulations change in the future.
Feel free to reuse this template for your PMP, and good luck for your Fundamentals of Software Project Management exam!
Here are the rules: do not panic, stay hydrated, and have fun!

CONTENTS

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1

PROJECT OVERVIEW

The shift to remote and hybrid work environments has led to an exponential rise in the use of video conferencing platforms such as Zoom, Google Meet, and Microsoft Teams. While these platforms enable seamless communication, they also produce vast amounts of unstructured conversational data that often goes underutilized. Existing solutions offer transcription and basic summaries but fall short in areas like persistent speaker identification, action item extraction, semantic search, and enterprise-level integration.

IntelliMeet Pro is designed to address these gaps by acting as an AI-powered meeting assistant that transforms raw conversations into structured, actionable knowledge. It integrates advanced natural language processing, persistent diarization, and semantic search with real-time transcription and summarization. Action items and decisions are automatically synchronized with popular project management tools such as Jira, Trello, and Asana, while Slack bot integration simplifies collaboration. With features like calendar synchronization, workspace management, bilingual transcription, and secure role-based access, IntelliMeet Pro not only enhances meeting efficiency but also ensures continuity, accountability, and improved organizational decision-making.

1.1 Research Plan

1.1.1 Scope

The scope of IntelliMeet Pro focuses on developing an AI-powered meeting assistant that automates the entire meeting management workflow. The system will cover automated participation in online meetings, real-time transcription, advanced speaker diarization with persistent identification, AI-driven summarization with action item extraction, semantic search across historical data, and seamless integration with project management tools. Additional features include Slack bot conversational support, real-time calendar synchronization, workspace management with role-based access control, and support for bilingual transcription (English and Urdu). Out-of-scope components include mobile application development, video processing, advanced business intelligence analytics, CRM integrations, and multi-tenant white-label deployments.

1.1.2 Purpose

The purpose of this project is to address the limitations of traditional meeting practices and existing transcription tools by providing a comprehensive solution that bridges the gap between communication and execution. Traditional note-taking often results in missed information, poor action tracking, and low searchability. Current AI-assisted platforms provide basic transcription or summaries but lack persistent diarization, semantic search, cross-platform automation, and enterprise-ready integrations. IntelliMeet Pro is designed to overcome these limitations by transforming raw conversations into structured, actionable knowledge, ensuring that organizational meetings directly contribute to productivity, collaboration, and informed decision-making.

1.1.3 Objectives

The key objectives of IntelliMeet Pro are as follows:

- Develop an autonomous AI bot capable of joining and recording scheduled meetings across Zoom, Google Meet, and Microsoft Teams without manual intervention.
- Implement advanced speaker diarization with persistent voice identification to maintain participant profiles across multiple meetings.
- Generate intelligent meeting summaries with automatic action item extraction and proper participant attribution.
- Provide semantic search functionality to allow natural language queries across all meeting transcripts and summaries.
- Enable seamless integration with project management tools such as Jira, Trello, and Asana for automatic synchronization of action items.
- Build a Slack bot to support conversational queries regarding meeting content and organizational knowledge.
- Deploy secure payment processing and subscription management with individual and enterprise tiers.
- Implement workspace management to support team collaboration through role-based access control and shared meeting histories.
- Support bilingual transcript translation (English and Urdu) to cater to diverse organizational needs.
- Deliver a responsive web application with real-time collaboration, intuitive dashboards, and historical navigation.

1.2 Assumptions and Constraints

1.2.1 Personnel

Assumptions

- The two student developers assigned to this project will remain consistently available and committed throughout the project timeline, with minimal disruptions due to academic workload or personal circumstances.

- The student team possesses or will acquire the necessary technical skills in AI integration, backend development, cloud storage, and API usage (e.g., OpenAI, Zoom, Google Meet, Azure) to deliver all planned functionality.
- University supervisors and advisors will be accessible for regular feedback, support, and guidance during key project milestones and deliverable reviews.
- Any required external technical consultation or mentoring (e.g., related to AI or system integration) will be available on time, informally, or via university resources.

Constraints

- Only two team members are available to execute the entire project, which limits parallel task execution and increases the dependency on effective time management.
- The team lacks access to full-time professional software developers or a QA team, placing limitations on testing depth and formal code reviews.
- No formal human resources (e.g., UI/UX designers, DevOps engineers) are available, which may restrict the project's ability to fully implement advanced or polished UI features and continuous deployment processes.

1.2.2 Timing

Assumptions

- The academic schedule will remain stable, and no significant calendar disruptions (e.g., strikes, extended holidays, institutional delays) will affect the planned delivery schedule.
- Development work can continue at a steady pace alongside other academic responsibilities and commitments.
- Milestones (e.g., mid-term submissions, project demos, supervisor reviews) will be respected and used as feedback loops to course-correct as needed.

Constraints

- The final version of the project must be completed and delivered no later than May 2026, aligning with the end of the academic term and FYP deadlines. No further extension is possible.
- Due to fixed academic timelines, any major deviation from the planned schedule (e.g., delayed API access, prolonged debugging cycles) could jeopardize timely project completion.
- Time for performance optimization, user testing, and documentation is limited and must be strictly scheduled to avoid last-minute rushes or missed academic criteria.

1.2.3 Budget

Assumptions

- The project will rely entirely on free-tier services, student licenses, open-source tools, and personal computing equipment, without incurring financial costs.

- Required services like OpenAI's API, Azure Blob Storage, Google Translate, Pinecone, and third-party integrations (Zoom, Slack, Jira, etc.) will either offer a free tier sufficient for project use or be accessible via student credits.
- No significant out-of-pocket expenses will be needed to procure hardware, software, or infrastructure during the course of the project.

Constraints

- There is no formal budget allocated for the project. This restricts the use of premium cloud services, high-volume APIs, or commercial software that may offer faster or more scalable functionality.
- All technology decisions (e.g., infrastructure, APIs, frameworks) must be made based on cost-efficiency and sustainability within free usage limits.
- The project cannot outsource any development work or purchase commercial tools for UI/UX, testing, or monitoring, which may impact feature richness or professional-grade polish.

1.3 Activities and Deliverables

Phase 1: Project Initiation & Planning

Activities:

- Gather requirements from stakeholders and potential users.
- Evaluate and select technology stack (APIs, cloud services, tools).
- Design system architecture and define module specifications.
- Create detailed project plan with milestones and work breakdown structure (WBS).

Deliverables:

- **Requirements & Design Document:** Contains functional & non-functional requirements, use cases, and technical specifications.
- **Project Plan:** Gantt chart, work breakdown structure, resource allocation, and timeline up to May 2026.

Phase 2: Core Infrastructure & Backend Setup

Activities:

- Set up backend services (user authentication, database design).
- Implement cloud infrastructure (Azure, Blob Storage, Key Vault, etc.).
- Integrate APIs for transcription, translation, and other services.
- Develop initial backend endpoints for data management.

Deliverables:

- **Infrastructure & API Integration Prototype:** Demonstrates backend API integration and cloud infrastructure setup.
- **Backend Core Module:** Working core system including authentication, data storage, and basic API endpoints.

Phase 3: AI / NLP Components

Activities:

- Implement speech-to-text transcription with speaker diarization.
- Develop summarization module for action items and key points extraction.
- Build semantic search functionality using Retrieval Augmented Generation (RAG) + vector DB.
- Integrate translation support (English - Urdu).

Deliverables:

- **AI/NLP Module:** Includes working transcription, speaker diarization, meeting summarization, search, and translation features.
- Meets performance targets: 90% transcription accuracy, speaker attribution, search speed, translation quality.

Phase 4: Integrations & Frontend Development

Activities:

- Integrate with project management tools (Jira, Trello, Asana).
- Develop Slack bot / conversational interface for user interaction.
- Build web frontend UI (dashboard, meeting playback, transcript viewer, history).
- Set up calendar sync and notification systems.

Deliverables:

- **Integrated Frontend + Tool Sync Module:** Enables synchronization of meeting action items and tasks with project management tools.
- **Slack Bot & Notification System:** Functional Slack bot for querying meeting data and notifications for upcoming tasks.

Phase 5: Payment, Subscription & Security Layers

Activities:

- Implement subscription tiers and payment processing (Stripe or another service).
- Develop role-based access control and secure authentication methods.
- Conduct security and privacy audits (data protection, encryption, access control).

Deliverables:

- **Subscription / Billing Module:** Includes at least three subscription tiers and a working payment processing system.
- **Security & Access Control Module:** Contains role-based access management, secure authentication, and encrypted data storage.

Phase 6: Testing, User Feedback, & Refinement

Activities:

- Conduct integration testing and unit testing for backend and frontend systems.
- Perform usability testing and gather feedback from test users.
- Optimize system performance (latency, response times).
- Address bugs and refine features based on feedback.
- Finalize documentation (User manual, API documentation, deployment guide).

Deliverables:

- **Test Report & Feedback Report:** Includes results from functional tests, performance testing, and usability feedback.
- **Final Product & Documentation:** Complete system, fully functional, with user and technical documentation.

Phase 7: Deployment & Closure

Activities:

- Deploy the system to a live environment or demonstrateable test environment.
- Perform final acceptance testing with stakeholders (including project supervisor).
- Deliver project presentation/demo to university and stakeholders.
- Conduct post-project review and document lessons learned.

Deliverables:

- **Deployed System & Demo:** Working system, accessible to users, demonstrating all core features.
- **Project Closure Report:** Includes project outcomes, lessons learned, and recommendations for future improvements.

CHAPTER 2

PROJECT CONTEXT

2.1 Process Model

The IntelliMeet Pro project will follow an Incremental and Iterative Process Model inspired by Agile principles. This approach has been chosen due to the complexity of AI/NLP integration, the need for continuous feedback, and the academic constraints of the project timeline. Instead of attempting to build the system as a single deliverable, the solution will be developed in small, manageable increments.

Each iteration will deliver a working module (e.g., transcription, diarization, summarization, integrations), enabling early testing, supervisor validation, and refinement. Core modules such as backend infrastructure and transcription services will be prioritized in earlier iterations, while advanced features such as Slack bot integration, billing, and role-based access will be developed in later cycles.

Key characteristics of the chosen process model include:

- Time-boxed sprints (3–4 weeks) aligned with academic checkpoints (mid-term reviews, demos).
- Frequent feedback loops with supervisors to ensure alignment with academic and project objectives.
- Parallel exploration of AI APIs and integration testing to mitigate risks early.
- Continuous integration and testing of modules to avoid technical debt accumulation.

This process model balances agility, adaptability, and structured delivery, ensuring the project remains on track despite limited resources.

2.2 Process Improvement Plan

To ensure the development process improves over time and adapts to challenges, the following strategies will be implemented:

1. Retrospectives at the end of each iteration – After every sprint, the team will reflect on what worked well, what challenges were faced, and what improvements are needed for the next cycle.

2. Coding and Documentation Standards – The team will adopt consistent naming conventions, Git branching workflows, and inline documentation to improve collaboration and code readability.

3. Incremental Testing Approach – Unit testing, integration testing, and usability checks will be incorporated into each cycle, rather than left for the end, improving defect detection and resolution.

4.Feedback-Driven Adjustments – Continuous input from supervisors, mock end-users, and peer reviewers will guide refinements in functionality and usability.

5. Risk Tracking – Potential issues (e.g., API usage limits, AI accuracy) will be logged, monitored, and addressed with contingency strategies (e.g., switching to alternative APIs).

By embedding improvement mechanisms into the process, the project ensures that productivity increases while risks and inefficiencies decrease over time.

2.3 Product Acceptance Plan

The acceptance of IntelliMeet Pro will be based on whether the final product satisfies the objectives, deliverables, and performance benchmarks defined in Chapter 1. Acceptance will follow a structured evaluation process:

- Acceptance Criteria:
  - Accurate transcription (90% accuracy for English, 85% for Urdu).
  - Correct and persistent speaker diarization across meetings.
  - Action item extraction and semantic search functionality validated through test scenarios.
  - Seamless integration with at least one project management tool (e.g., Trello).
  - Functioning Slack bot responding to user queries about meetings.
  - Secure role-based access and subscription management for at least two tiers.
  - Responsive web interface with intuitive navigation.
- Testing & Validation:
  - Unit and integration testing by the developers.
  - Supervisor-led acceptance testing to validate functionality against academic requirements.
  - User acceptance testing (UAT) with selected peers to assess usability, collaboration, and real-time response.
- Documentation & Delivery:
  - Final project report, user manual, and deployment guide will accompany the product.
  - Demonstration during the final FYP evaluation will serve as the formal acceptance checkpoint.

Successful fulfillment of these criteria will lead to formal acceptance by the supervisor and evaluation committee.

2.4 Infrastructure, tools and methods

The IntelliMeet Pro project relies on a set of open-source tools, free-tier cloud services, and student-licensed software. The infrastructure, tools, and methods are divided as follows:

- Infrastructure
  - Personal laptops of developers (Windows/Linux environments).
  - Cloud storage and compute services:
    - Azure Blob Storage (document storage).
    - Azure Key Vault (secure key management).
  - Deployment environment:
    - Free-tier hosting or university-provided servers.
- Development Tools
  - Backend: Node.js / Python (for AI/NLP integration and API development).
  - Frontend: React.js (for dashboards and web interfaces).
  - Database: MongoDB Atlas (cloud-based, free-tier) for storing:
    - Transcripts.
    - User data.
    - Workspace management.
  - AI/NLP:
    - OpenAI API (summarization, action item extraction).
    - Azure Cognitive Services (speech-to-text).
    - Google Translate (bilingual support).
- Collaboration & Project Management Tools
  - GitHub for version control and issue tracking.
  - Trello for internal sprint planning and task management.
  - Slack for communication and integration testing.
- Methods
  - Agile-inspired iterative development with 3–4 week sprints.
  - Test-driven development (TDD) for critical components like transcription and summarization.
  - Continuous integration using GitHub Actions.

This combination of infrastructure and tools ensures cost-efficiency while supporting the technical and organizational needs of the project.

2.5 Project Organization

The project will be executed by a two-person student development team, under the guidance of faculty supervisors. The organization structure is as follows:

- Project Manager / Developer
  - Oversees project planning, scheduling, and reporting.
  - Leads backend development, database design, and integration with external APIs.
  - Responsible for testing, documentation, and ensuring milestone completion.
- Lead Developer / Research Engineer
  - Focuses on AI/NLP implementation, including transcription, summarization, diarization, and semantic search.
  - Handles frontend development (React.js dashboards, web app UI).
  - Coordinates Slack bot integration and usability testing.
- Faculty Supervisor
  - Provides academic oversight, feedback, and evaluation at key milestones.
  - Ensures the project aligns with FYP objectives and grading criteria.
- Advisor / External Mentor
  - Provides domain expertise in AI, cloud integration, or software architecture.
  - Advises on best practices and advanced features.

This lean project organization reflects the constraints of limited personnel while ensuring clear role distribution. Collaborative communication and structured responsibilities will be key to successful delivery.

CHAPTER 3

PROJECT PLANNING

3.1 Work Activities

The IntelliMeet Pro project will be developed across seven major phases, each containing structured activities and deliverables as outlined in the project proposal. These phases ensure systematic progress, modular development, and continuous validation.

Phase 1: Project Initiation & Planning

- Gather requirements from supervisors, stakeholders, and mock end-users.
- Identify functional and non-functional requirements.
- Select technology stack (APIs, cloud services, frameworks).
- Design system architecture with backend, frontend, and AI/NLP modules.
- Define module specifications and prepare Work Breakdown Structure (WBS).

Phase 2: Core Infrastructure & Backend Setup

- Set up backend services including authentication and database schema.
- Configure cloud infrastructure (Azure Blob Storage, Key Vault).
- Establish API connections for transcription and translation.
- Develop baseline endpoints for data management.
- Deliverables: Infrastructure Prototype, Backend Core Module.

Phase 3: AI/NLP Components

- Implement speech-to-text transcription with speaker diarization.
- Develop summarization module for extracting action items and key highlights.
- Build semantic search with vector DB + RAG (Retrieval Augmented Generation).
- Integrate bilingual support (English Urdu).

- Deliverables: AI/NLP Module with validated accuracy benchmarks.

Phase 4: Integrations & Frontend Development

- Integrate project management tools (Jira, Trello, Asana).
- Develop Slack bot for conversational queries.
- Build web frontend UI including dashboard, transcript viewer, and meeting history navigation.
- Implement calendar synchronization and notifications.
- Deliverables: Integrated Frontend + Tool Sync, Slack Bot & Notification System.

Phase 5: Payment, Subscription & Security Layers

- Implement subscription tiers with payment processing (Stripe/free alternatives).
- Develop role-based access control and secure authentication.
- Apply encryption to sensitive data.
- Conduct internal security audits.
- Deliverables: Subscription & Billing Module, Security & Access Control Module.

Phase 6: Testing, User Feedback & Refinement

- Perform unit, integration, and usability testing.
- Collect supervisor and peer feedback.
- Optimize system performance (reduce latency, improve search speed).
- Fix bugs and refine usability.
- Finalize documentation (User Manual, API docs, Deployment Guide).
- Deliverables: Test Report & Feedback Report, Refined Product with Documentation.

Phase 7: Deployment & Closure

- Deploy to a live/demo environment.
- Conduct supervisor-led acceptance testing.
- Deliver final presentation and demo to stakeholders.
- Compile closure report with lessons learned.
- Deliverables: Deployed System, Demo, Project Closure Report.

These structured activities ensure that development progresses in incremental, testable stages, while aligning with academic deliverables and deadlines.

3.2 Time Allocation

The project follows a 9-month academic timeline (Sept 2025 – May 2026). Each phase is assigned time blocks with consideration for academic deadlines, workload balance, and feedback cycles.

Phase	Timeline	Duration	Key Milestones
Phase 1: Initiation & Planning	Sept 2025 – Early Oct 2025	˜4 weeks	Proposal defense, finalized syste architecture, project plan submission
Phase 2: Core Infrastructure & Backend Setup	Mid Oct – Nov 2025	˜6 weeks	Backend prototype (authentication, DB cloud setup)
Phase 3: AI/NLP Components	Dec 2025 – Mid Jan 2026	˜6 weeks	Working transcription, diarizatio summarization, semantic search
Phase 4: Integrations & Frontend Development	Mid Jan – Feb 2026	˜5 weeks	Trello/Slack integration, functiona frontend dashboard
Phase 5: Payment, Subscription & Security	Early Mar 2026	˜3 weeks	Subscription system + role-based acces control
Phase 6: Testing & Refinement	Mid Mar – Mid Apr 2026	˜4 weeks	Usability testing, performanc optimization, documentation
Phase 7: Deployment & Closure	Late Apr – May 2026	˜3 weeks	Final deployment, supervisor acceptance FYP presentation/demo

Key Considerations for Time Allocation:

- Parallel Work: Where possible, AI/NLP research will run in parallel with backend setup to save time.
- Buffer Time: Each phase includes 1 week buffer for unexpected delays (API limits, debugging, academic load).
- Supervisor Checkpoints: Mid-term and pre-final demos will act as validation gates for progress.
- Academic Deadlines: Final product delivery is non-negotiable by May 2026; therefore, all development activities must conclude by mid-April to leave room for deployment and presentation preparation.

By allocating time in structured phases, while embedding buffers and checkpoints, the plan ensures feasibility within the given academic timeline.

CHAPTER 4

PROJECT ASSESSMENT AND CONTROL

4.1 Requirements Management Plan

The IntelliMeet Pro project involves a mix of functional (transcription, diarization, summarization, semantic search, integrations) and non-functional requirements (accuracy, usability, security, scalability). To ensure all requirements are properly captured, tracked, and validated, the following plan will be adopted:

- **Requirement Gathering:** Initial requirements have been captured from the project proposal, supervisors, and mock users. These will be reviewed and refined iteratively.
- **Documentation:** All requirements will be documented in the Requirements & Design Document. Functional requirements will include use cases, while non-functional requirements will include measurable targets (e.g., 90% transcription accuracy).
- **Tracking Tool:** Requirements will be tracked in Trello using dedicated cards linked to milestones. GitHub Issues will also be used for traceability with code.
- **Validation:** Each requirement will be validated via supervisor reviews, interim demos, and user acceptance testing (UAT).
- **Change Handling:** Any new or modified requirement will be logged, analyzed for feasibility (time, resources), and only accepted if aligned with scope and deadlines.

This plan ensures that requirements remain visible, testable, and aligned with project objectives throughout the development cycle.

4.2 Scope Change Control Plan

Given the academic timeline and limited resources, managing scope creep is critical. The plan for scope control includes:

- **Baseline Scope:** Defined in Chapter 1 (objectives, features, deliverables). Out-of-scope items (mobile app, CRM integration, white-label deployment) are explicitly excluded.
- **Change Request Process:** Any proposed scope change (e.g., adding a new integration) must be formally documented in a Change Request Form, including justification, expected benefit, and estimated impact on time/budget.
- **Approval Authority:** The project supervisor will serve as the final authority to approve or reject scope changes. Only changes with significant academic or functional benefit will be considered.
- **Impact Assessment:** For approved changes, the team will update the WBS, Gantt chart, and resource allocation plan.
- **Scope Monitoring:** Regular checkpoints with supervisors will ensure that work remains aligned with the baseline scope.

This controlled approach balances innovation with discipline, preventing uncontrolled feature expansion while keeping the project achievable.

4.3 Schedule Control Plan

The fixed academic deadline (May 2026) requires strict schedule management. The following control mechanisms will be applied:

- **Baseline Schedule:** Established in Chapter 3 (phased timeline with milestones).
- **Monitoring:** Weekly progress reviews with the team and bi-weekly check-ins with the supervisor. Tasks will be tracked via Trello boards and updated with status (To Do, In Progress, Done).
- **Milestone Reviews:** Each phase completion will include a short demo or report to validate progress against the timeline.
- **Variance Analysis:** Any delays will be analyzed for root cause (technical issues, academic workload). Corrective actions (rescheduling tasks, prioritizing critical modules) will be applied.
- **Buffer Time:** Each phase includes a one-week buffer for unforeseen delays, helping maintain overall schedule integrity.

By embedding checkpoints and buffers, the project ensures timely completion without last-minute rush.

4.4 Budget Control Plan

Since no formal budget has been allocated, budget control focuses on optimizing free-tier and student-license resources.

- **Baseline Budget Assumptions:** The project will rely on free services (GitHub, Trello, Slack, MongoDB Atlas, Azure free credits, OpenAI trial/student credits). No out-of-pocket costs are expected.

- **Budget Monitoring:** API usage will be monitored (e.g., OpenAI tokens, Azure transcription limits). Weekly checks will ensure usage remains within free-tier limits.
- **Contingency Plan:** If usage exceeds free-tier quotas, the team will:
  - Switch to alternative open-source APIs (e.g., Whisper for transcription).
  - Reduce usage intensity during testing.
  - Apply for additional student credits where possible.
- **Budget Control Authority:** The project team jointly manages costs; any potential paid feature requires supervisor approval.

This plan ensures cost-effectiveness while preventing unexpected expenses.

4.5 Quality Assurance Plan

The IntelliMeet Pro system must meet academic standards and deliver a reliable, user-friendly solution. The QA plan includes:

- **Standards & Guidelines:**
  - Coding standards: consistent naming, modular design, Git branching strategy.
  - Documentation standards: inline comments, API documentation, user manuals.
- **Testing Strategy:**
  - Unit Testing: For individual modules (transcription, summarization).
  - Integration Testing: For API interactions and system workflows.
  - System Testing: End-to-end validation of functionality.
  - User Acceptance Testing (UAT): Supervisor and peer review for usability.
- **Quality Metrics:**
  - Transcription accuracy $\geq 90\%$ (English), $\geq 85\%$ (Urdu).
  - Response latency ≤ 3 seconds for queries.
  - Successful diarization for $\geq 80\%$ of test cases.
- **Defect Management:** Bugs will be logged in GitHub Issues, prioritized, and resolved in subsequent sprints.

By combining technical testing with user validation, the project ensures a balance between performance, usability, and reliability.

4.6 Dissemination Plan

The dissemination of IntelliMeet Pro involves sharing results and outcomes with both academic and general audiences.

4.6.1 Scientific dissemination

- **Final Year Project Report:** A comprehensive academic document containing research methodology, design, implementation, evaluation, and results.
- **Supervisor & Examiner Presentations:** Formal presentations and demos as part of academic evaluation.
- **Technical Documentation:** Detailed system architecture diagrams, API references, and deployment guides for future researchers or students.
- **Potential Conference Submission:** Depending on results, a research paper may be prepared for a student-level conference or symposium on AI and software engineering.

4.6.2 General-purpose dissemination

– **Live Demonstration:** Project showcase during the FYP exhibition to faculty, peers, and external visitors.

– **Website / Web App:** Deployment of a responsive web application for demonstration purposes.

– **Video Demo:** A short recorded walkthrough highlighting system features and use cases.

– **Portfolio & Online Sharing:** Documentation of the project on GitHub and LinkedIn to demonstrate technical skills and project outcomes.

This dual dissemination approach ensures the project contributes to academic research knowledge while also showcasing practical value to the broader audience.

CHAPTER 5

SUPPORTING PROCESS PLANS

5.1 Risk Management

Risk management is essential for ensuring the successful completion of IntelliMeet Pro within the academic timeline and institutional constraints. Risks are identified, categorized as negative (threats) and positive (opportunities), and strategies are defined to mitigate or leverage them. A continuous monitoring approach will be adopted, with regular reviews during milestone presentations and supervisor meetings.

5.1.1 Negative risks

Negative risks represent potential threats that could hinder the project's progress or quality. Key risks include:

1. Technical Complexity

- **Description:** Integrating advanced features like AI-based recommendations and automatic scheduling could exceed the team's current expertise.
- **Mitigation:** Incremental development using prototyping; consulting supervisors for technical guidance; adopting open-source libraries where possible.

2. Time Management Issues

- **Description:** The academic calendar (two semesters) and parallel coursework may reduce the effective time available for development.
- **Mitigation:** Use of a detailed Gantt chart and Agile sprints to allocate weekly tasks; buffer time reserved before evaluations.

3. Resource Constraints

- **Description:** Limited access to testing environments or specialized hardware may slow down development.

- **Mitigation:** Utilize cloud-based tools (GitHub, Firebase, Google Calendar APIs); simulate environments instead of relying on physical hardware.

4. Requirement Creep

- **Description:** Continuous requests for additional features (e.g., multi-platform compatibility) could dilute focus.
- **Mitigation:** Strict scope control through supervisor approval; maintain a backlog of enhancements for post-FYP development.

5. Team Coordination Issues

- **Description:** As the team consists of two developers balancing coursework and project tasks, misalignment can occur.
- **Mitigation:** Weekly coordination meetings; version control via GitHub; clear division of responsibilities.

5.1.2 Positive risks

Positive risks are opportunities that could enhance project value if leveraged effectively.

1. Early Completion of Core Features

- **Opportunity:** If main modules (scheduling, reminders, dashboard) are completed earlier, time can be invested in polishing UI/UX or adding advanced analytics.
- **Strategy:** Keep a list of "stretch goals" such as multi-language support or machine learning–based recommendations.

2. Adoption Beyond FYP

- **Opportunity:** The project may gain interest from other departments, opening doors for institutional adoption or external funding.
- **Strategy:** Maintain scalability in architecture; prepare polished documentation and deployment guides.

3. Skill Development

- **Opportunity:** Mastery of tools like React, Firebase, and API integration will increase employability and future opportunities for the developers.
- **Strategy:** Document learnings, contribute to open-source communities, and highlight outcomes in CVs/portfolios.

4. Integration with University Systems

- **Opportunity:** Successful prototype could be integrated into the university's official portal.
- **Strategy:** Design with modular APIs so that IntelliMeet Pro can connect with existing LMS or calendar systems.

5.2 Cost Management

Although IntelliMeet Pro is an academic project with no direct financial investment, cost management is still important in terms of time, effort, and resource utilization. The cost management plan focuses on minimizing indirect costs while maximizing productivity.

1. Direct Costs

- Software development does not require paid licenses since open-source frameworks (React, Node.js) and free tiers (Firebase, GitHub) are utilized.
- Cloud hosting or domain purchases, if required for deployment, will be handled under minimal student-friendly pricing or university-provided resources.

2. Indirect Costs

- Time is the most valuable resource. Inefficient task execution could delay milestones, increasing "opportunity cost."
- To minimize this, strict adherence to the schedule and Agile sprints will ensure time-efficient development.

3. Cost Monitoring

- A weekly progress review will track whether additional resources (paid APIs, hosting, or tools) are required.
- Any expenditure above free-tier services will be approved jointly by the project supervisor and team.

4. Effort Allocation as Cost Equivalent

- Since labor is unpaid (student effort), cost is represented by estimated hours per activity (from Chapter 3).
- For example, development of the scheduling module is allocated approximately 60 hours, while UI design is around 40 hours.
- This allocation ensures effort-based cost management.

5. Sustainability Consideration

- Post-FYP, if IntelliMeet Pro is adopted at scale, hosting and maintenance costs will emerge.
- Cost management now ensures the project remains lightweight and affordable, ready for future scalability.

.

ACRONYMS

FSPM Fundamentals of Software Project Management. ii

PMP Project Management Plan. ii

REFERENCES

[1] Project Management Institute, ed. *A guide to the project management body of knowledge / Project Management Institute*. Sixth edition. PMBOK guide. Newtown Square, PA: Project Management Institute, 2017. ISBN: 978-1-62825-184-5.

[2] Kathy Schwalbe. *Information technology project management*. Revised Seventh edition. OCLC: ocn864657391. Australia: Cengage Learning, 2014. ISBN: 978-1-285-84709-2.